Perception for Action in Latent World Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

Perception for action suggests that representations of the world should be shaped not by visual fidelity alone, but by their relevance for actions. At the same time, latent JEPA-style world models advocate learning compact predictive states from high-dimensional observations to facilitate the prediction of future states, but end-to-end training of these models is nontrivial because representations may collapse if our only goal is to construct a latent state that is easy to predict. We show that a suitable inverse dynamics regularization addresses both issues: it is an effective anti-collapse mechanism that induces action-aligned representations. By forcing latent states to preserve information about the action underlying a transition, it biases the model toward the controllable degrees of freedom of the environment while discarding uncontrollable distractors. This yields stable latent world models trained end-to-end from offline, reward-free trajectories, without frozen encoders, exponential moving averages, or complex latent regularizers. Empirically, the learned latent spaces are compact, interpretable, and enable competitive planning performance across simple 2D and 3D control tasks.

1 Introduction

World models that predict future states given actions are central to intelligent agents [1–3]. Learning such models directly in pixel space is difficult because observations are high-dimensional and pixel-level reconstruction encourages the model to capture background details that are often irrelevant for control. Latent world models avoid these issues by learning dynamics in a representation space rather than in observation space [2, 4–6], and more recently, Joint Embedding Predictive Architectures (JEPAs) [7–9] dispense with reconstruction altogether and predict directly in embedding space.

JEPA-style world models from pixels are appealing but susceptible to *representation collapse*: when the encoder and the latent dynamics model are trained jointly to make correct predictions in embedding space, the encoder may learn to map all observations to a constant embedding, making the dynamics prediction task trivial while rendering the learned model useless for other tasks [10, 11]. Recent work approaches this problem from several angles. DINO-WM [12] sidesteps the problem by freezing a pretrained encoder. PLDM [13] adds a multi-term variance–covariance regularizer on the embeddings. LeWorldModel [14, 15] matches the embedding distribution to an isotropic Gaussian via SIGReg. V-JEPA 2 [16] uses stop-gradient targets from an exponential-moving-average target encoder.

While representation collapse is an important practical problem, it also points to the basic question of what a useful world model representation should retain. It has been argued that this connects to causality: a causal representation [17] should not merely encode predictive correlations, but also actions and their effects, to provide an interventional world model [3]. In cognitive psychology and neuroscience, the term *perception for action* captures the view that perception ultimately subserves action [18]. *Common coding* proposes that perceived events and planned actions share a unified representational domain, with actions represented partly in terms of their anticipated perceptual

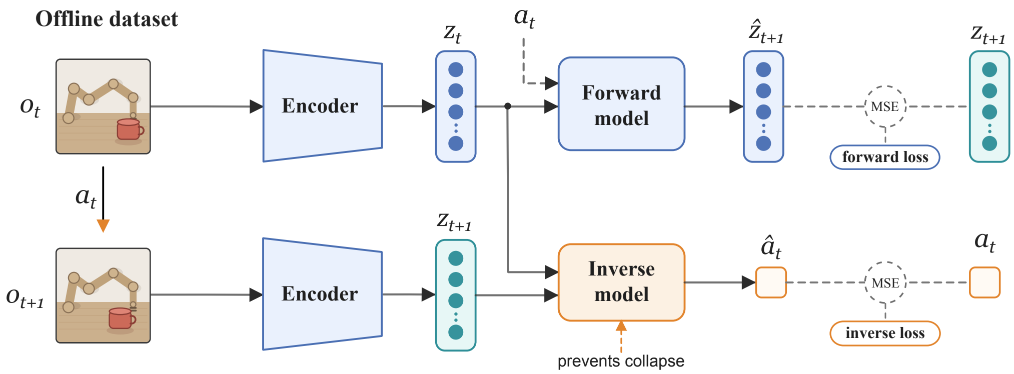

Figure 1: **Method overview.** We train an encoder f_θ, a forward dynamics model g_ϕ, and an inverse dynamics model h_ψ jointly from an offline dataset of transitions (o_t, a_t, o_{t+1}). The encoder maps each observation to a compact embedding, $z_t = f_\theta(o_t)$ and $z_{t+1} = f_\theta(o_{t+1})$. The forward model predicts the next embedding from the current embedding and action, $\hat{z}_{t+1} = g_\phi(z_t, a_t)$, and is supervised by the mean-squared forward loss $\mathcal{L}_{\text{fwd}}$ between $\hat{z}_{t+1}$ and z_{t+1}. In parallel, the inverse model predicts the executed action from the two embeddings, $\hat{a}_t = h_\psi(z_t, z_{t+1})$, supervised by the mean-squared inverse loss $\mathcal{L}_{\text{inv}}$ between $\hat{a}_t$ and a_t. Both losses backpropagate into the encoder through the joint objective $\mathcal{L} = \mathcal{L}_{\text{fwd}} + \lambda\, \mathcal{L}_{\text{inv}}$. The inverse loss acts as an anti-collapse mechanism: to enable action recovery, f_θ must preserve the action-relevant information contained in (o_t, o_{t+1}).

effects [19], rendering the content of a representation dependent on the organism's action repertoire.[1] Representations should thus not only be judged by how much information they preserve about observations, but by whether they preserve the distinctions that matter for the actions available to the agent and for the consequences of those actions.

Enactive perception is the theory that perception is not a passive reception of sensory input, but an active bodily exploration of the world [22]. A recent instantiation of this view focuses on *sensorimotor contingencies* as the lawful relationships between an agent's actions and the resulting changes in sensory input [23]. With this in mind, we return to our practical problem, i.e., collapse: *imagine we can train an embedding to respect those contingencies, by forcing it to retain information sufficient to reconstruct the agent's actions.* Could this help avoid representational collapse?

We use a simple and principled mechanism to explore this: *inverse dynamics regularization*. We add a single-step inverse dynamics head that predicts the action taken between two consecutive observations from their embeddings, and we propagate gradients from this head into the encoder, as illustrated in Fig. 1. The intuition is direct: if two consecutive embeddings contain enough information to recover the action that produced the transition, they must preserve features of the observations that are relevant for controllable dynamics. This encourages the encoder to retain action-relevant information while discarding the rest. Unlike distributional regularizers that prescribe the geometry of the embedding space, inverse dynamics anchors the representation to a task-grounded quantity—the action.

Setting and contributions. We consider offline datasets of trajectories consisting of video frames and corresponding continuous actions, without rewards, task labels, or knowledge of the data-collecting policy. This setting broadly covers demonstration data, action-annotated video datasets, and logged interaction data. Our contributions are:

1. a simple latent world model trained end-to-end from pixels, using inverse dynamics regularization as the sole anti-collapse mechanism with one additional hyperparameter,
2. evidence that the learned representations are not merely non-collapsed, but track the controllable degrees of freedom, preserve spatial geometry, and filter out uncontrollable distractors,
3. competitive planning performance against SIGReg-regularized baselines across 2D and 3D control tasks, showing that this structural advantage carries over to downstream control.

[1] This also relates to Gibson's notion of *affordances*: organisms perceive the environment in terms of possibilities for action, such as grasping, walking, avoiding, or manipulating [20]. Going further back, von Uexküll's concept of the *Umwelt* describes the organism-specific world disclosed by a creature's sensory and motor capacities [21]. Ultimately, the world should thus look rather different to different creatures, even if they appear to inhabit the same environment.

2 Background and related work

Latent world models. Learning dynamics models in learned latent spaces has a rich history. The Dreamer family [2, 4, 24] and PlaNet [25] learn latent dynamics with a pixel decoder under a variational objective. TD-MPC [5, 6] drops the decoder and trains a joint encoder and latent dynamics model end-to-end with a temporal-difference objective. JEPAs [7–9] push this further, dispensing with both reconstruction and reward signal and predicting directly in embedding space. Our work belongs to the JEPA family and operates in the offline, reward-free regime.

Representation collapse. The self-supervised learning literature has charted a design space of remedies for representation collapse: contrastive losses with negative samples [26, 27], asymmetric architectures with stop-gradients [10, 28], and regularizers on the embedding distribution [11, 29]. Recent JEPA world models inherit choices from this design space: DINO-WM [12] freezes a pretrained encoder, PLDM [13] adds multiple auxiliary loss terms including variance–covariance regularization, temporal smoothness, and inverse dynamics, LeWorldModel [14, 15] matches embeddings to an isotropic Gaussian, and V-JEPA 2 [16] uses stop-gradient and exponential moving average target networks. We share the JEPA training framework and the offline pixel-action setting with these works but use inverse-dynamics as the standalone anti-collapse mechanism, as described in Sec. 3.

Inverse dynamics. The use of inverse dynamics for representation learning is not new. Pathak et al. [30] use it to learn features that capture controllable aspects of the environment for intrinsic-reward computation. A line of theoretical work shows that, in finite rich-observation settings with exogenous distractors, multi-step inverse objectives can recover control-relevant latent state information under certain assumptions [31–34]. While none of these results transfers to our continuous-state, continuous-action setting with a single-step inverse, we treat them as motivation for using action prediction as an inductive bias toward controllable state structure. Concurrent work has begun to combine inverse dynamics with world-model objectives in related but distinct settings [35–37]; we discuss our differences from each of these works in Sec. F.

Other directions. Several adjacent lines share architectural ingredients but address different problems. Action-free or latent-action world models infer pseudo-actions from video and use them to supervise dynamics: LAPO [38] and DynaMo [39] use a learned inverse dynamics model to extract latent actions from action-free demonstrations, and APV [40] pretrains an action-free latent video predictor for downstream action-conditional world-model learning. Visual RL methods share encoders between auxiliary objectives and policies but do not learn explicit world models [41–43]. Bisimulation metrics offer a complementary lens on which states should map to similar embeddings [44, 45], and causal abstraction [46] formalizes a related consistency between structural models via a commutative diagram on states and interventions. Sec. F provides further detail.

3 Method

Dataset and setting. We assume access to an offline dataset $\mathcal{D} = \{(o_t, a_t, o_{t+1})\}$ of transitions, where $o_t, o_{t+1} \in \mathcal{O}$ are consecutive observations (e.g., video frames) and $a_t \in \mathcal{A} \subseteq \mathbb{R}^m$ is a continuous action taken in between. We do not require a behavior policy or a reward function. Our goal is to learn an encoder $f_\theta : \mathcal{O} \to \mathcal{Z}$ that maps observations to a compact embedding space $\mathcal{Z} \subseteq \mathbb{R}^d$, together with a forward dynamics model $g_\phi : \mathcal{Z} \times \mathcal{A} \to \mathcal{Z}$ that predicts the next embedding from the current embedding and action.

Forward dynamics in embedding space. Given a transition (o_t, a_t, o_{t+1}), we encode both observations and train the dynamics model to match the embedding of the next observation,

$$z_t = f_\theta(o_t), \quad z_{t+1} = f_\theta(o_{t+1}), \quad \hat{z}_{t+1} = g_\phi(z_t, a_t), \tag{1}$$

by minimizing the mean-squared forward loss

$$\mathcal{L}_{\text{fwd}} = \mathbb{E}_{(o_t,a_t,o_{t+1})\sim\mathcal{D}} \left[\|\hat{z}_{t+1} - z_{t+1}\|_2^2\right]. \tag{2}$$

This objective alone admits a trivial solution: if f_θ maps every observation to a constant embedding $z^\star$ and g_ϕ outputs $z^\star$ regardless of its inputs, then $\mathcal{L}_{\text{fwd}} = 0$. Such a collapsed representation carries no information about the observation and is useless for downstream planning or control.

Inverse dynamics regularization. To prevent this collapse, we introduce an inverse dynamics model $h_\psi : \mathcal{Z} \times \mathcal{Z} \to \mathcal{A}$ that predicts the action taken between two consecutive observations from their embeddings, $\hat{a}_t = h_\psi(z_t, z_{t+1})$, trained with the mean-squared action loss

$$\mathcal{L}_{\text{inv}} = \mathbb{E}_{(o_t,a_t,o_{t+1})\sim\mathcal{D}} \left[\|\hat{a}_t - a_t\|_2^2\right]. \tag{3}$$

Under squared loss, a collapsed encoder would give the inverse model identical inputs for every transition, so the best inverse predictor is a constant action, with risk $\mathbb{E}\|a_t - \mathbb{E}[a_t]\|_2^2$. Any reduction below this constant-predictor risk requires (z_t, z_{t+1}) to preserve information that is predictive of a_t. Such a nontrivial inverse prediction thus rules out a fully collapsed representation. In contrast to distributional priors such as SIGReg's isotropic-Gaussian regularizer [14], this mechanism does not prescribe the geometry of the embedding space: it only requires that the encoder preserve whatever information about a_t is contained in the pair (o_t, o_{t+1}).

Joint training objective. We jointly optimize the encoder, forward model, and inverse model by minimizing

$$\mathcal{L} = \mathcal{L}_{\text{fwd}} + \lambda\,\mathcal{L}_{\text{inv}}, \tag{4}$$

where $\lambda > 0$ controls the strength of the inverse dynamics regularization. The forward loss updates f_θ and g_ϕ; the inverse loss updates f_θ and h_ψ. The encoder, therefore, receives gradients from both objectives, producing embeddings that are simultaneously predictive under forward dynamics and action-informative under inverse dynamics.

4 Learned latent structure

To build intuition for what the inverse-dynamics-regularized objective of Eq. (4) learns, and to assay which structure of the world the resulting representations reflects, we instantiate the method on a minimal testbed with known ground-truth state, intrinsic dimension, and action geometry.

A controlled testbed. We consider a *dot world*: a single-dot environment whose state is fully described by the 2D position (x, y) of a colored dot rendered on a 64×64 white canvas; the observation $o_t \in \mathbb{R}^{3\times64\times64}$ is the rendered image. Actions $a_t = (\Delta x, \Delta y) \in \mathcal{A} \subset \mathbb{R}^2$ displace the dot, so the dynamics are $s_{t+1} = s_t + a_t$ in the state space. The dataset $\mathcal{D}$ is constructed by sampling, for each transition, a position and a random displacement. The encoder f_θ is a small CNN with the latent dimension $d = 64$, and the forward g_ϕ and inverse h_ψ models are two-layer MLPs. The exact architecture, dataset, and training details are deferred to Sec. E.

Recovered intrinsic dimension. First, we show that f_θ identifies the true 2D structure of the world from pixels and actions alone. Fig. 2 reports the PCA spectrum of the embeddings $\{f_\theta(o)\}$ over a uniform grid of dot positions. Two principal components carry essentially all the variance in the embedding space, and the spectrum drops sharply at the true intrinsic dimension $d_{\text{true}} = 2$; the remaining 62 directions are effectively collapsed. Projecting the embeddings onto the top two principal components reveals that the latent geometry is not only low-dimensional but also *spatially faithful*: the grid of world states maps to a near-square grid in latent space. A mild deformation occurs only near the boundary, which corresponds to out-of-distribution states. As a result, spatial neighbors in the observation space remain neighbors in the latent space. The encoder thus recovers an approximately 2-dimensional, topology-preserving representation of the state space without any ground-truth state supervision.

Forward model. The encoder and forward model should jointly satisfy a consistency condition: encoding the observation reached after taking action a should agree with first encoding the current observation and then applying $g_\phi(\cdot, a)$,

$$f_\theta(o_{t+1}) \stackrel{!}{=} g_\phi(f_\theta(o_t), a_t). \tag{5}$$

In other words, the diagram on the left of Fig. 3 must commute [46, 47]. To test it, we encode an initial observation o_1, unroll g_ϕ autoregressively along a 5-step action sequence $a_1, \ldots a_5$, and compare the predicted embeddings against the encoded ground-truth embeddings $z_t = f_\theta(o_t)$. The predicted and encoded trajectories agree closely along the entire rollout (Fig. 3, right), illustrating that Eq. (5) approximately holds on the trained model. Each $a_t \in \mathcal{A}$ induces a corresponding displacement $\tilde{a}_t$ in

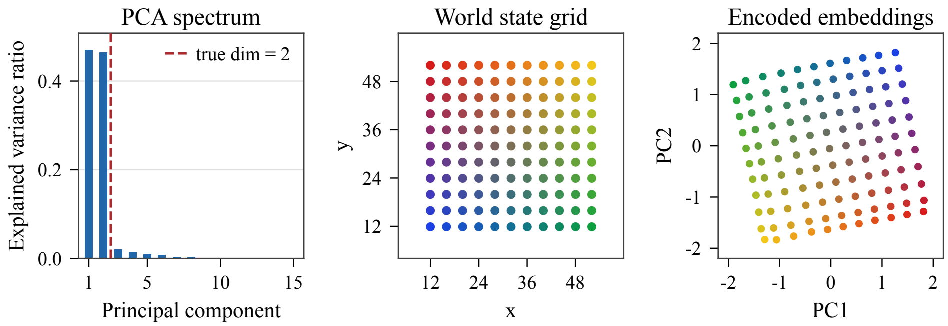

Figure 2: **Dot world latent geometry. Left:** PCA spectrum of the learned embeddings; the explained-variance ratio drops sharply past the true intrinsic dimension $d_{\text{true}} = 2$ (red dashed line). **Center:** grid of probe world states (x, y), color-coded by position. **Right:** the same probes embedded by f_θ and projected onto the top two principal components. Despite no state supervision, the encoder recovers an effectively 2-dimensional representation inside a 64-dimensional ambient latent space. Moreover, the recovered representation is spatially faithful to the true state space. The latent grid appears rotated relative to (x, y), as expected: PC axes are identifiable only up to rotation and sign.

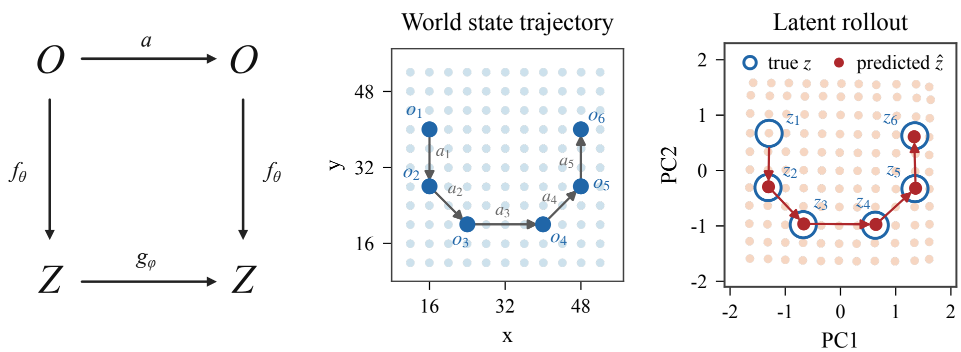

Figure 3: **Encoder and forward model commute. Left:** commutative diagram that must be satisfied by the learned representation: $f_\theta \circ a = g_\phi(\cdot, a) \circ f_\theta$. **Center:** a 5-step trajectory in world-state space with actions $a_1, \ldots, a_5$. **Right:** the corresponding rollout in latent space; predictions $\hat{z}_t$ (filled red) obtained by autoregressive application of g_ϕ track the encoded ground-truth embeddings $z_t = f_\theta(o_t)$ (open blue) along the entire rollout. Therefore, the encoder and forward model approximately commute, and world actions act as translations in latent space.

the relevant subspace of the latent space, so the forward model behaves as $g_\phi(z, a) \approx z + \tilde{a}$. The trained model thus delivers, in addition to the state representation f_θ, a representation of actions as latent translations $g_\phi(\cdot, a)$ —without any term in $\mathcal{L}$ enforcing such a structure.

Controllable degrees of freedom. The 2D dot world establishes that the learned representation has the right intrinsic dimension when it equals the action dimension. To stress-test this property, we vary the controllable degrees of freedom and add uncontrollable distractors. Fig. 4 reports four configurations. *Independent* contains two dots controlled by independent displacements, giving a 4D state and a 4D action. *Coupled* contains two dots that move together under a single 2D displacement. *Distractor* contains one controlled dot together with a second dot that moves randomly on its own and is not controlled by the action. *Combined* merges all three: two independent dots, one coupled pair, and one distractor, with a 6D action vector.

In every case, the number of significant principal components matches the controllable dimension of the system, with the remaining directions effectively collapsed. One may notice that the spectra within the controllable subspace are not exactly uniform, even though the state space is uniformly

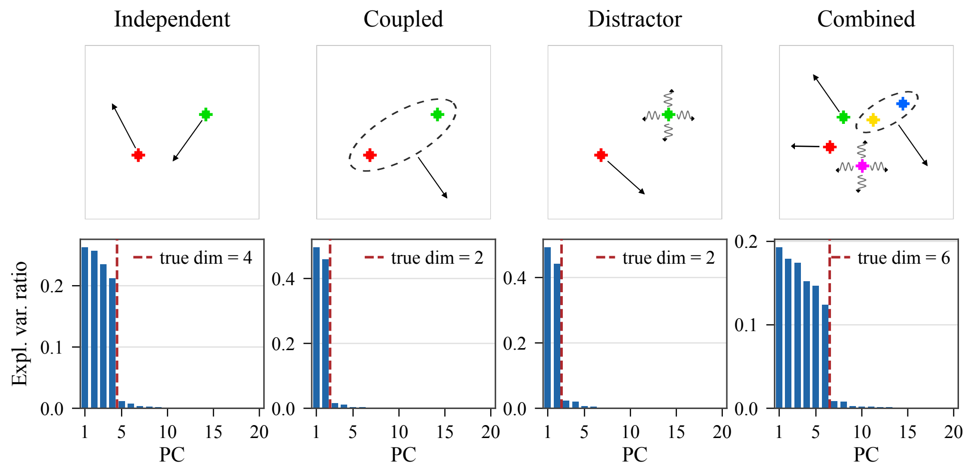

Figure 4: **Effective latent dimension tracks controllable degrees of freedom. Top row:** four dot-world configurations with controllable dimensions 4, 2, 2, and 6; in *Distractor* and *Combined*, the wavy-arrowed dot moves randomly and is not controlled by the action. **Bottom row:** PCA spectra of the corresponding learned embeddings, with the true intrinsic dimension marked by the red dashed line. The encoder allocates significant variance to exactly the controllable degrees of freedom and filters out uncontrollable distractors, correctly recovering a minimal necessary subspace inside the higher-dimensional ambient latent space.

covered under uniform coordinate sampling. However, this is expected: the encoder is free to rescale individual dimensions and concentrate variance unevenly, so what matters is the rank of the embedding, not the uniformity of its spectrum. Of the four configurations, the distractor case is particularly informative: even though the random dot's position varies in the observation, the encoder ignores it. This is the expected outcome of Eq. (4): a distractor's state is neither needed to recover the action from a transition (it is independent of a_t) nor predictable from past latent state under g_ϕ (since it evolves stochastically), so encoding it would only inflate $\mathcal{L}_{\text{fwd}}$ without reducing $\mathcal{L}_{\text{inv}}$.

Together, these experiments show that the inverse-dynamics-regularized objective produces representations that are low-dimensional, spatially faithful, and structure-preserving with respect to the action-induced dynamics.

5 Planning in latent space

The latent structure recovered above is only useful insofar as it carries over to richer environments and supports actionable downstream control.

Setup. We perform goal-conditioned trajectory optimization directly in the learned latent space, following the planning setup of LeWM [14]. Given a current observation o_1 and a goal observation o_g, we encode both with the frozen encoder, $z_1 = f_\theta(o_1)$ and $z_g = f_\theta(o_g)$, and roll the forward model out autoregressively from $\hat{z}_1 = z_1$, $\hat{z}_{t+1} = g_\phi(\hat{z}_t, a_t)$, to predict the latent trajectory induced by a candidate action sequence $a_{1:H}$ over a planning horizon H. Each candidate is scored by the terminal goal-matching cost

$$\mathcal{C}(a_{1:H}) = \|\hat{z}_{H+1} - z_g\|_2^2, \tag{6}$$

which we minimize with the Cross-Entropy Method (CEM) [48]: at each iteration, CEM samples action sequences from a Gaussian, scores them via latent rollouts under g_ϕ, and refits the sampling distribution to the top-scoring fraction. Because autoregressive rollouts accumulate prediction error as H grows, we wrap CEM in a receding-horizon model-predictive control (MPC) loop, executing only the first H optimized actions before re-encoding the resulting observation and replanning. Throughout, f_θ and g_ϕ remain frozen; only the action sequence is optimized.

Environments. We evaluate on four tasks shown in Fig. 5: TwoRoom (2D navigation), Reacher (continuous control), Push-T (2D contact-rich manipulation), and OGBench-Cube (3D tabletop manipulation). Datasets and environment details are deferred to Sec. B.

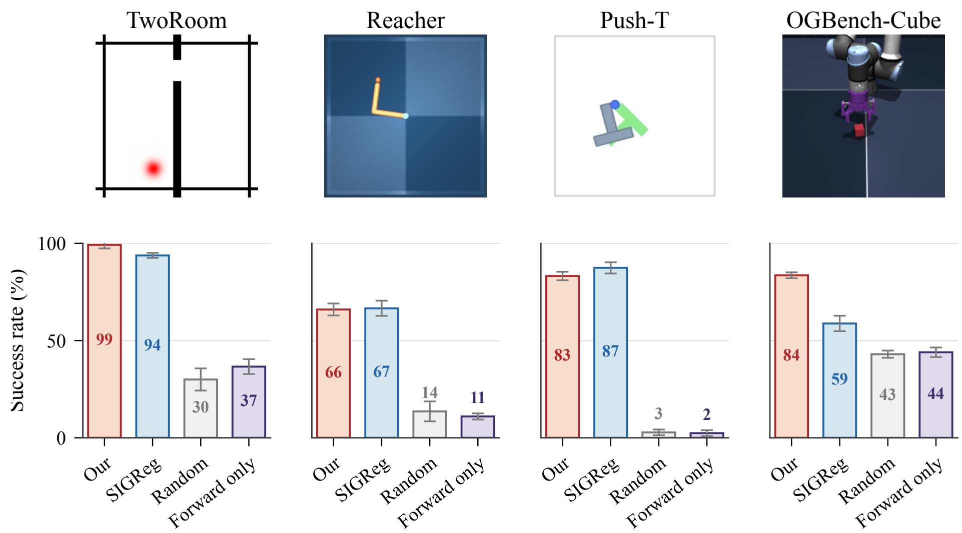

Figure 5: **Planning success across environments. Top:** the four evaluation environments—TwoRoom (2D navigation), Reacher (continuous control), Push-T (2D contact-rich manipulation), and OGBench-Cube (3D tabletop manipulation). **Bottom:** goal-conditioned planning success rate (mean and standard error over five seeds) under a fixed budget of 50 environment steps and a goal placed 25 steps ahead of the initial state. Inverse-dynamics regularization matches the SIGReg baseline on the three 2D tasks and clearly outperforms it on OGBench-Cube. Both regularized variants dominate the *Forward-only* ablation and the *Random*-action baseline, confirming that an effective anti-collapse mechanism is necessary for usable planning.

Baselines. We compare against three baselines. `SIGReg` [14] is our main head-to-head: it shares our encoder and predictor but replaces $\mathcal{L}_{\text{inv}}$ with a Gaussian-matching regularizer on the latent embeddings. `Forward-only` ($\lambda = 0$) ablates the regularizer entirely, isolating the contribution of our anti-collapse mechanism. `Random` samples actions uniformly from the action space and serves as a floor reference. Full implementation details are in the appendix.

Architecture. We adopt LeWM's encoder and predictor [14]: the encoder is a ViT-Tiny ($\sim$5M parameters) whose final CLS token is projected to $z \in \mathbb{R}^d$ with $d = 192$, and g_ϕ is a small transformer ($\sim$10M parameters) that predicts $\hat{z}_{t+1}$ from a single (z_t, a_t) pair, with a_t injected via AdaLN-zero. Our only architectural addition is the inverse model h_ψ: a 2-layer MLP of width 256 mapping $[z_t;\, z_{t+1}] \in \mathbb{R}^{2d}$ to $\hat{a}_t \in \mathbb{R}^m$. Details are provided in Sec. A.

Planning performance. All methods are evaluated with a fixed budget of 50 environment steps per episode and goals placed 25 steps ahead of the initial state. Both regularized variants substantially outperform Forward-only and Random across all four environments (Fig. 5). Our method approximately matches SIGReg on the three 2D tasks—TwoRoom, Reacher, and Push-T—and clearly outperforms it on OGBench-Cube, the only environment with a 3D contact-rich manipulation, where our method retains 84% success while SIGReg drops to 59%.

Probing physical state. To assess what physical content the embeddings retain, we train linear and two-layer MLP probes on frozen embeddings to regress the ground-truth state of each environment; held-out R^2 values are reported in Tab. 1.

Both regularized methods recover the underlying state essentially perfectly under MLP probes, and their linear-probe scores stay near saturation across most quantities, with our method holding a consistent linear advantage on the Push-T agent position and on all three Cube quantities, and SIGReg holding a corresponding advantage on the Reacher joint angles and the Push-T block pose. The Forward-only ablation is markedly poorer, particularly under linear probes, but is rarely fully degenerate—some residual variance survives even in a partially collapsed encoder. Both regularized representations therefore make the underlying physical state recoverable.

Table 1: **Probing the physical state.** Held-out R^2 for linear and two-layer MLP probes regressing ground-truth physical quantities from frozen embeddings; the best score per row and probe type is in bold. Both regularized methods (Our and SIGReg) recover the underlying physical state essentially perfectly under MLP probes.

Environment	Quantity	Linear probe			MLP probe		
		Our	SIGReg	Fwd.-only	Our	SIGReg	Fwd.-only
TwoRoom	agent position	**1.000**	0.996	0.832	**1.000**	1.000	0.991
Reacher	joint angles	0.946	**0.999**	0.647	0.995	**1.000**	0.962
Push-T	agent position	**0.993**	0.954	0.293	**0.988**	0.986	0.433
	block position	0.946	**0.973**	0.352	0.981	**0.996**	0.623
	block orientation	0.664	**0.931**	0.202	0.882	**0.987**	0.491
Cube	cube position	**0.998**	0.975	0.706	**0.998**	0.993	0.854
	gripper position	**0.999**	0.986	0.902	**0.999**	0.995	0.979
	gripper opening	**0.989**	0.961	0.269	**0.992**	0.983	0.427

Latent geometry. We next turn to the geometry that organizes them. Fig. 6 reports, for each environment, the PCA spectrum of held-out embeddings (top), the distribution of a representative ground-truth quantity in physical state space (middle), and the embeddings projected onto a 2- or 3-dimensional PC subspace (bottom). In every environment, the spectrum drops sharply, confirming that the embeddings concentrate on an effectively low-dimensional subspace of the 192-dimensional ambient latent space. The corresponding SIGReg visualizations are provided in Sec. D; while SIGReg also prevents collapse, its latent geometry appears less consistently organized by the controllable coordinates.

In TwoRoom, the encoder behaves much as in the dot world: PC_1 and PC_2 recover a spatially faithful Cartesian map of the agent, and the wall—a region unreachable to the agent—leaves an empty band in latent space, providing further evidence that the recovered representation is spatially faithful. In Push-T, the agent position is again encoded in $(\mathrm{PC}_1, \mathrm{PC}_2)$, but the spectrum has heavier tails: the embedding must additionally accommodate the position and orientation of the manipulable block (cf. Tab. 1). Reacher reveals a richer geometry. Its state space is the torus $\mathbb{T}^2$ of two periodic angles (θ_1, θ_2), which cannot be continuously encoded into $\mathbb{R}^2$ due to a discontinuity at $\theta = 0 \sim 2\pi$. A natural representation is the product-of-circles embedding $(\theta_1, \theta_2) \mapsto (\cos\theta_1, \sin\theta_1, \cos\theta_2, \sin\theta_2)$, which lifts $\mathbb{T}^2$ into $\mathbb{R}^4$. Its coordinate 3D projections are circular cylinders, which is exactly the signature we observe in the encoded embeddings: the spectrum exposes four appreciable components, and every 3D PC subspace in Fig. 6 (bottom) shows a clear cylindrical shape. The two angles, however, spread across all four components, which is consistent with arbitrary linear mixing of the product-of-circles coordinates. Finally, in Cube the gripper position is recovered in $(\mathrm{PC}_1, \mathrm{PC}_3)$; PC_2 and the heavier tail of the spectrum carry the cube pose and gripper opening (cf. Tab. 1).

Takeaway. Across all four environments, inverse-dynamics regularization recovers compact latent spaces whose dimension and geometry mirror controllable structure: Cartesian degrees of freedom form approximately linear directions, while periodic variables form circles. The structure carries over to 2D and 3D control, with the largest planning gain over SIGReg appearing on the 3D environment.

6 Discussion

We showed that inverse dynamics regularization addresses two coupled issues simultaneously. First, as an anti-collapse mechanism, a single-step inverse dynamics head suffices to stabilize end-to-end training of a JEPA-style latent world model from pixels, while introducing only one hyperparameter and imposing no distributional prior on the embedding space. Second, the same mechanism biases the encoder toward a compact, action-sufficient representation due to the joint pressure from the forward and inverse dynamics loss terms. Empirically, this pressure yields representations that track the controllable degrees of freedom of the environment, preserve both Cartesian and periodic state geometry, and filter out uncontrollable distractors, although no term in the objective explicitly enforces this structure. The learned representation further supports latent planning that matches

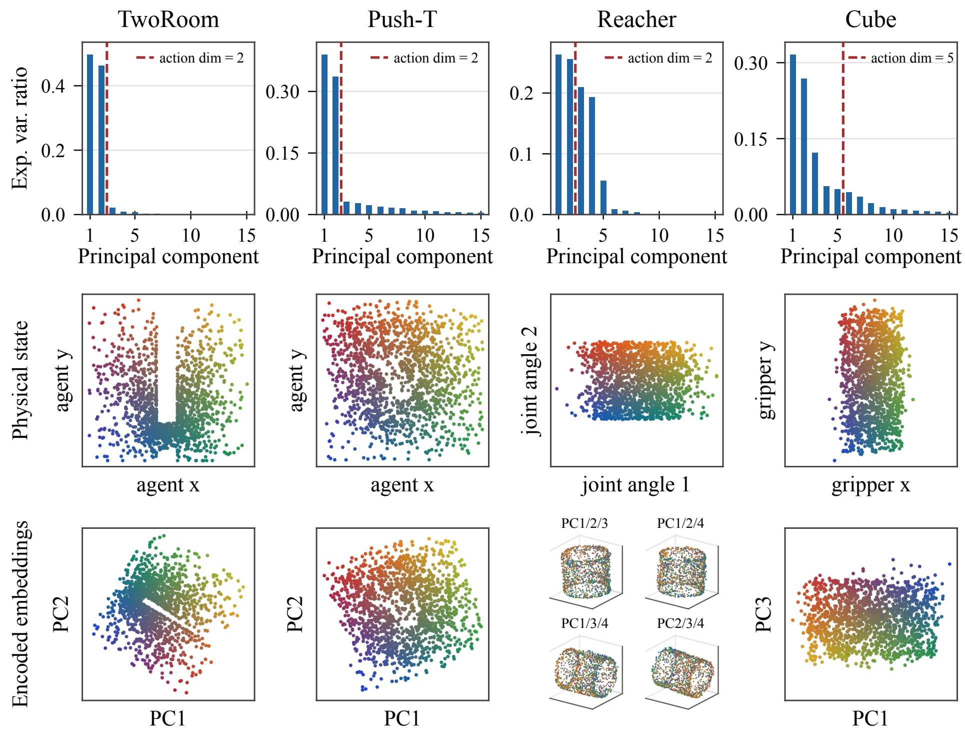

Figure 6: **Latent geometry of inverse-regularized embeddings.** For each environment we show the PCA spectrum of held-out embeddings (top), the distribution of a representative ground-truth quantity in physical state space (middle), and the embeddings projected onto a 2- or 3-dimensional PC subspace, color-coded by the same physical quantity (bottom). The dashed red lines mark the action dimension. Across all four environments, the embeddings concentrate on a low-dimensional subspace whose geometry mirrors the controllable state space: Cartesian coordinates (TwoRoom and Push-T agent, Cube gripper) appear as approximately linear directions, while the periodic joint angles in Reacher are encoded as a flat 2-torus, visible as a cylindrical structure in any 3D PC projection.

or outperforms a strong SIGReg-regularized baseline across four control tasks, with the largest margin on the most complex 3D manipulation environment with a five-dimensional action space. These results support the view of inverse dynamics regularization as more than a collapse-prevention mechanism: it instantiates a "perception for action" bias in latent world models which is effective and simple. We consider this a good thing: the fact that the above can be achieved with such a simple objective (Eq. (4)) makes us optimistic for the future prospect of this approach.

Limitations and future work. Our analysis assumes that actions are recoverable from consecutive observations, which may fail when distinct actions induce the same visible change. Relatedly, because the encoder currently maps individual frames to latent states, it will not capture quantities that are not identifiable from a single frame, such as velocities, even when these quantities would improve forward prediction; incorporating short observation histories into the encoder or predictor is a natural extension. Furthermore, the action-aligned bias, while desirable, may discard information needed for downstream tasks that depend on aspects of the environment irrelevant to the training actions (we consider this a feature rather than a bug). In addition, biased behavior policies may create action-correlated but uncontrollable distractors, which could be captured by the encoder. Finally, our planning experiments inherit the usual limitations of offline world models: limited dataset coverage constrains the reachable plans, long-horizon rollouts remain vulnerable to compounding model error, and our empirical validation is restricted to moderate-scale simulated control tasks. Future work should study more robust data regimes, multi-frame, and multi-step inverse objectives. Finally, while we use inverse loss only as a regularizer, inverse models themselves could be integrated into latent or hierarchical planners to reduce reliance on long autoregressive forward rollouts.

References

[1] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[2] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.

[3] Bernhard Schölkopf. Causality for machine learning. 2019. URL `http://arxiv.org/abs/1911.10500`. Published in: Probabilistic and Causal Inference: The Works of Judea Pearl.

[4] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[5] Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *ICML*, 2022.

[6] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. *ICLR*, 2024.

[7] Yann LeCun. A path towards autonomous machine intelligence. *OpenReview*, 2022.

[8] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023.

[9] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-jepa: Latent video prediction for visual representation learning. *arXiv preprint arXiv:2402.04252*, 2024.

[10] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent: A new approach to self-supervised learning. In *NeurIPS*, 2020.

[11] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.

[12] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, pages 79115–79135. PMLR, 2025. URL `https://proceedings.mlr.press/v267/zhou25t.html`.

[13] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL `https://neurips.cc/virtual/2025/poster/116649`.

[14] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. URL `https://arxiv.org/abs/2603.19312`.

[15] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025. URL `https://arxiv.org/abs/2511.08544`.

[16] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL `https://arxiv.org/abs/2506.09985`.

[17] B. Schölkopf, F. Locatello, S. Bauer, N. R. Ke, N. Kalchbrenner, A. Goyal, and Y. Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021.

[18] Melvyn A. Goodale and A. David Milner. Separate visual pathways for perception and action. *Trends in Neurosciences*, 15(1):20–25, 1992.

[19] Wolfgang Prinz. A common coding approach to perception and action. In Odmar Neumann and Wolfgang Prinz, editors, *Relationships Between Perception and Action: Current Approaches*, pages 167–201. Springer, Berlin, 1990.

[20] James J. Gibson. *The Ecological Approach to Visual Perception*. Houghton Mifflin, 1979.

[21] Jakob von Uexküll. *Streifzüge durch die Umwelten von Tieren und Menschen: Ein Bilderbuch unsichtbarer Welten*, volume 21 of *Verständliche Wissenschaft*. Verlag von Julius Springer, Berlin, 1934.

[22] Francisco J. Varela, Eleanor Rosch, and Evan Thompson. *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press, Cambridge, MA, 1991.

[23] J. Kevin O'Regan and Alva Noë. A sensorimotor account of vision and visual consciousness. *Behavioral and Brain Sciences*, 24(5):939–1031, 2001.

[24] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering Atari with discrete world models. In *ICLR*, 2021.

[25] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.

[26] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. In *arXiv preprint arXiv:1807.03748*, 2018.

[27] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020.

[28] Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *CVPR*, 2021.

[29] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *ICML*, 2021.

[30] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *ICML*, 2017.

[31] Yonathan Efroni, Dipendra Misra, Akshay Krishnamurthy, Alekh Agarwal, and John Langford. Provably filtering exogenous distractors using multistep inverse dynamics. In *International Conference on Learning Representations (ICLR 2022)*, 2022. URL `https://openreview.net/forum?id=RQLLzMCefQu`.

[32] Alex Lamb, Riashat Islam, Yonathan Efroni, Aniket Rajiv Didolkar, Dipendra Misra, Dylan J Foster, Lekan P Molu, Rajan Chari, Akshay Krishnamurthy, and John Langford. Guaranteed discovery of control-endogenous latent states with multi-step inverse models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL `https://openreview.net/forum?id=TNocbXm5MZ`.

[33] Riashat Islam, Manan Tomar, Alex Lamb, Yonathan Efroni, Hongyu Zang, Aniket Didolkar, Dipendra Misra, Xin Li, Harm van Seijen, Remi Tachet des Combes, and John Langford. Agent-controller representations: Principled offline RL with rich exogenous information. *arXiv preprint arXiv:2211.00164*, 2022.

[34] Zakaria Mhammedi, Dylan J. Foster, and Alexander Rakhlin. Representation learning with multi-step inverse kinematics: An efficient and optimal approach to rich-observation RL. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 24659–24700. PMLR, 2023. URL `https://proceedings.mlr.press/v202/mhammedi23a.html`. Oral presentation.

[35] Yuci Han and Alper Yilmaz. Enhancing policy learning with world-action model. *arXiv preprint arXiv:2603.28955*, 2026. URL `https://arxiv.org/abs/2603.28955`.

[36] Basile Terver, Randall Balestriero, Megi Dervishi, David Fan, Quentin Garrido, Tushar Nagarajan, Koustuv Sinha, Wancong Zhang, Mike Rabbat, Yann LeCun, and Amir Bar. A lightweight library for energy-based joint-embedding predictive architectures. *arXiv preprint arXiv:2602.03604*, 2026. URL `https://arxiv.org/abs/2602.03604`.

[37] Jiacan Yu, Siyi Chen, Mingrui Liu, Nono Horiuchi, Vladimir Braverman, Zicheng Xu, Dan Haramati, and Randall Balestriero. Why and how auxiliary tasks improve JEPA representations. In *UniReps: 3rd Edition of the Workshop on Unifying Representations in Neural Models*, 2025. URL `https://openreview.net/forum?id=ZVx4SdKhlc`.

[38] Dominik Schmidt and Minqi Jiang. Learning to act without actions. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=rvUq3cxpDF`.

[39] Zichen Jeff Cui, Hengkai Pan, Aadhithya Iyer, Siddhant Haldar, and Lerrel Pinto. Dynamo: In-domain dynamics pretraining for visuo-motor control. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL `https://openreview.net/forum?id=vUrOuc6NR3`.

[40] Younggyo Seo, Kimin Lee, Stephen L. James, and Pieter Abbeel. Reinforcement learning with action-free pre-training from videos. In *Proceedings of the 39th International Conference on Machine Learning (ICML 2022)*, volume 162 of *Proceedings of Machine Learning Research*, pages 19561–19579. PMLR, 2022. URL `https://proceedings.mlr.press/v162/seo22a.html`.

[41] Michael Laskin, Aravind Srinivas, and Pieter Abbeel. Curl: Contrastive unsupervised representations for reinforcement learning. In *International conference on machine learning*, pages 5639–5650. PMLR, 2020.

[42] Ilya Kostrikov, Denis Yarats, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. *arXiv preprint arXiv:2004.13649*, 2020.

[43] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Reinforcement learning with prototypical representations. In *International Conference on Machine Learning*, pages 11920–11931. PMLR, 2021.

[44] Norm Ferns, Prakash Panangaden, and Doina Precup. Metrics for finite Markov decision processes. In *UAI*, 2004.

[45] Pablo Samuel Castro. Scalable methods for computing state similarity in deterministic Markov decision processes. In *AAAI*, 2020.

[46] P. K. Rubenstein*, S. Weichwald*, S. Bongers, J. M. Mooij, D. Janzing, M. Grosse-Wentrup, and B. Schölkopf. Causal consistency of structural equation models. In *Proceedings of the Thirty-Third Conference on Uncertainty in Artificial Intelligence (UAI)*, 2017. URL `http://auai.org/uai2017/proceedings/papers/11.pdf`.

[47] Heinrich Hertz. *The Principles of Mechanics Presented in a New Form*. Macmillan, London, 1899.

[48] Reuven Y Rubinstein. Optimization of computer simulation models with rare events. *European Journal of Operational Research*, 99(1):89–112, 1997.

[49] Lucas Maes, Quentin Le Lidec, Dan Haramati, Nassim Massaudi, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel-v1: Reproducible world modeling research and evaluation. *arXiv preprint arXiv:2602.08968*, 2026.

[50] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *7th Robot Learning Workshop: Towards Robots with Human-Level Abilities*, 2025. URL `https://openreview.net/forum?id=jON7H6A9UU`.

[51] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=M992mjgKzI.

[52] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

[53] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.

[54] Alexander Levine, Peter Stone, and Amy Zhang. Multistep inverse is not all you need. *Reinforcement Learning Journal*, 2:884–925, 2024. URL https://rlj.cs.umass.edu/2024/papers/Paper117.html. Presented at the Reinforcement Learning Conference (RLC 2024).

[55] David Brandfonbrener, Ofir Nachum, and Joan Bruna. Inverse dynamics pretraining learns good representations for multitask imitation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d36dfcdb14473a8526111c221660f2ab-Abstract-Conference.html.

[56] Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *ICLR*, 2021.

A Implementation details

A.1 Training objective

Alg. 1 gives PyTorch-style pseudocode for the mini-batch objective used to train the latent world model. The encoder receives gradients from both the forward prediction loss and the inverse dynamics loss, which is what prevents representation collapse.

Algorithm 1 Mini-batch loss for inverse-dynamics-regularized latent world model training.

```
def loss(o_t, a_t, o_tp1, lambda_inv=10.0):
    """
    o_t, o_tp1: (B, C, H, W)  consecutive pixel observations
    a_t:        (B, A)        action taken between them
    lambda_inv: (float)       inverse dynamics loss weight
    """
    z_t   = encoder(o_t)                    # (B, D)
    z_tp1 = encoder(o_tp1)                  # (B, D)

    z_hat = fwd_model(z_t, a_t)             # predict next embedding
    a_hat = inv_model(z_t, z_tp1)           # predict intervening action

    loss_fwd = F.mse_loss(z_hat, z_tp1)     # forward dynamics loss
    loss_inv = F.mse_loss(a_hat, a_t)       # inverse dynamics loss

    return loss_fwd + lambda_inv * loss_inv
```

A.2 Architecture

We adopt the encoder and forward model of LeWorldModel [14] essentially unchanged, and add a small inverse head h_ψ. Inputs are 224×224 RGB images.

Encoder f_θ. A Vision Transformer Tiny (ViT-Tiny) with patch size 14, instantiated from the Hugging Face library. The final CLS token is projected to $z \in \mathbb{R}^d$ with $d = 192$; the encoder has ≈ 5M parameters.

Forward model g_ϕ. A ViT-S backbone with learned positional embeddings and causal masking over the observation history, matching the LeWM predictor; the action a_t is injected via AdaLN-zero. However, in contrast to LeWM, we set the history length to 1 on *every* environment, so g_ϕ predicts $\hat{z}_{t+1}$ from a single (z_t, a_t) pair, as described in Sec. 3. LeWM instead uses history 3 on Push-T and OGBench-Cube. The forward model has ≈ 10M parameters.

Inverse model h_ψ. Our only architectural addition. A 2-layer MLP of width 256 with ReLU activations, mapping $[z_t;\, z_{t+1}] \in \mathbb{R}^{2d}$ to $\hat{a}_t \in \mathbb{R}^m$, without output activation. Approximately 0.1M parameters.

A.3 Optimization details

All world models are trained end-to-end with the objective in Eq. (4). We use the same optimization settings for our method, the forward-only ablation, and the SIGReg baseline; only the regularization term is changed. For our method, the inverse-dynamics weight λ is environment-specific. The values are listed in Tab. 2.

Table 2: **Inverse-dynamics weights.** Values of λ in $\mathcal{L}_{\text{fwd}} + \lambda\mathcal{L}_{\text{inv}}$.

Environment	λ
TwoRoom	0.1
Reacher	5
Push-T	30
OGBench-Cube	1

The remaining training hyperparameters are shared across environments unless explicitly noted in Tab. 3. We train on two-frame snippets with a single-step forward target. Since datasets are loaded with frameskip 5, the action encoder and inverse head receive the concatenated action over the skipped interval. The SIGReg comparison uses the same optimizer and architecture, with $\lambda = 0$ and SIGReg weight 0.09.

Table 3: **Optimization hyperparameters.**

Hyperparameter	Value
Optimizer	AdamW
Learning rate	10^{-4}
Weight decay	10^{-3}
Batch size	256
Training epochs	10
Gradient clipping	1.0

Choosing λ. Fig. 7 shows that λ requires some environment-level tuning. For example, $\lambda = 0.1$ is optimal for TwoRoom but causes Reacher to collapse to near-random planning performance. The final values in Tab. 2 were chosen from this broad sweep. The sweep contains one trained model per λ value, so the shaded bands show binomial standard error over the 100 evaluation episodes rather than variation across training seeds.

A.4 Planning protocol

For each evaluation episode, the policy receives the current observation o_t and a goal observation o_g, encodes them as $z_t = f_\theta(o_t)$ and $z_g = f_\theta(o_g)$, and optimizes candidate action sequences with CEM under the terminal cost in Eq. (6). Candidate sequences are rolled out using g_ϕ for H latent model steps, where each model-step action is a block of five primitive environment actions.

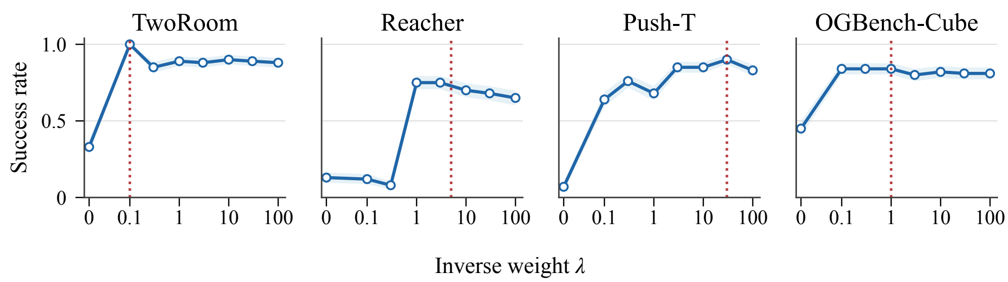

Figure 7: **Sensitivity to inverse-dynamics weight.** Goal-conditioned planning success rate as a function of the inverse-dynamics loss weight λ at goal offset 25. Each panel corresponds to one environment, and the red dotted line marks the value used for the main paper experiments.

Planning uses the same MPC and CEM hyperparameters in all four environments (Tab. 4). We sample 100 start states without replacement from the held-out evaluation dataset, requiring that the same episode contains a goal frame 25 steps later. The environment is reset to the sampled start state, the target is set from the goal row, and success is measured over a budget of 50 primitive environment steps. The sampled tasks are fixed across methods and seeds within each environment.

Table 4: **Planning hyperparameters.**

Hyperparameter	Value
Planning horizon H	5 model steps
Executed steps K	5 model steps
Action block	5 primitive actions
CEM population	300
CEM iterations	30
CEM elites	30
Initial CEM variance scale	1.0
Evaluation tasks	100
Goal offset	25 primitive steps
Evaluation budget	50 primitive steps

A.5 Probing protocol.

For Tab. 1, we freeze f_θ, encode embeddings $z = f_\theta(o)$, and train probes on $25{,}000$ training embeddings per environment and method, reporting held-out R^2 on a disjoint $5{,}000$-sample validation split. The linear probe is ridge regression with $\alpha = 10^{-3}$. The nonlinear probe is a two-layer MLP with widths $256, 128$ and ReLU activations. For vector-valued targets, including sine–cosine angle encodings, we report uniformly averaged R^2 over target coordinates.

A.6 Compute.

All training and planning runs used a single NVIDIA H100 GPU. Each world model was trained in under 4 hours.

B Environments and datasets

Our experiments use the four environments and offline datasets adopted by Maes et al. [14] (Fig. 8). All models are trained and evaluated through the `stable-worldmodel` codebase [49], which standardizes the environment interfaces, dataset loading, and downstream planning procedure.

For every environment, we use an episode-level $90/10$ train/validation split: f_θ, g_ϕ, and h_ψ are trained on the training episodes, while all reported metrics—planning (Figs. 5 and 9), probes (Tab. 1), and latent-geometry analyses—are computed on the held-out validation split.

(a) TwoRoom. TwoRoom [13, 50] is a 2D point-navigation task with a simple constrained geometry: the agent starts in one room and the goal lies in the other, with a single doorway as the only passage between them. The script moves the agent first toward the doorway and then toward the goal after crossing the wall. The dataset contains 10,000 noisy scripted rollouts, averaging 92 steps.

(b) Push-T. Push-T is a 2D contact-rich manipulation task in which a dot-shaped agent must push a T-shaped object into a target pose. We use the DINO-WM expert demonstrations [12], consisting of 20,000 episodes with mean length 196.

(c) OGBench-Cube. OGBench-Cube [51] is a tabletop manipulation task where a robot gripper must move a cube to a specified target pose. We use the single-cube setting and the benchmark's scripted dataset: 10,000 trajectories, each of length 200.

(d) Reacher. Reacher is the two-link arm task from the DeepMind Control Suite [52]. We follow the DINO-WM variant, where success is defined by matching the target joint configuration, not just by end-effector proximity. The offline dataset contains 10,000 SAC-generated episodes [53], each 200 steps long.

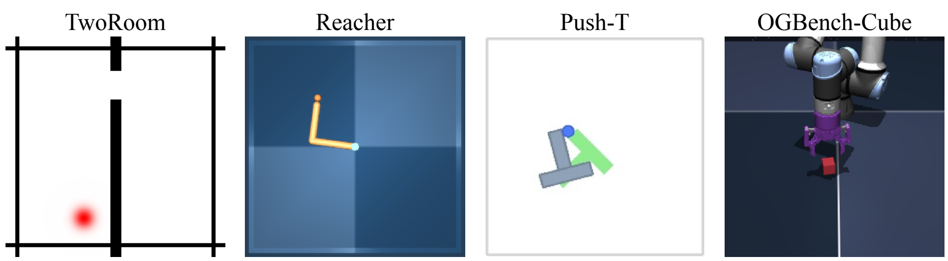

Figure 8: **Environments.** Four evaluation environments—TwoRoom (2D navigation), Reacher (continuous control), Push-T (2D contact-rich manipulation), and OGBench-Cube (3D tabletop manipulation).

C Planning performance on extended goal offsets

Fig. 9 evaluates whether the learned latent dynamics remain useful when the goal is placed farther from the initial state. For each goal offset, the start and goal observations are sampled from the held-out evaluation split, and the planner is given a budget of $2\times$ the offset in primitive environment steps. Thus the evaluation becomes harder because the model must support longer latent rollouts. The same trained models and MPC/CEM planner from Fig. 5 are used throughout.

The inverse-regularized model is stable across longer horizons on TwoRoom and OGBench-Cube, where SIGReg either degrades sharply or remains consistently lower. On Reacher, the inverse and SIGReg curves stay close over the tested offsets. Push-T is the main failure case: both regularized methods drop as the offset increases.

D SIGReg latent geometry

Fig. 10 repeats the representation summary of Fig. 6 for the SIGReg baseline. The same held-out embeddings and PCA protocol are used, with the bottom row showing $(\mathrm{PC}_1, \mathrm{PC}_2)$ for each environment. Unlike the inverse-dynamics-regularized embeddings, the SIGReg spectra do not exhibit a clear elbow. This is expected: the SIGReg regularizer explicitly encourages the embedding distribution to match an isotropic Gaussian, spreading variance across many principal components rather than concentrating it in a compact task-relevant subspace. As a result, although state information remains recoverable by probes, the resulting representation isn't low-dimensional, and is less directly interpretable than the inverse-dynamics-regularized representation.

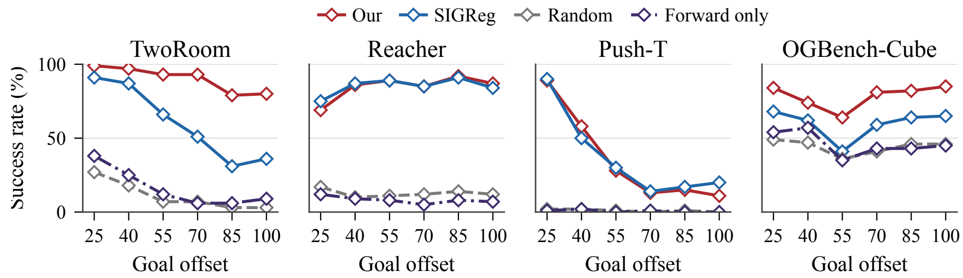

Figure 9: **Robustness to planning horizon.** Goal-conditioned planning success rate as a function of *goal offset*, the number of environment steps between the initial and goal observations; the planner's evaluation budget is fixed at $2\times$ the goal offset. Methods and environments match Fig. 5. Our method maintains near-flat success across horizons on TwoRoom and OGBench-Cube, while SIGReg either degrades sharply (TwoRoom) or trails consistently (Cube). The two regularized methods are comparable on Reacher and Push-T.

E Dot world details

E.1 Environment

Each observation is a 64×64 RGB image rendered on a white canvas: each dot is drawn as a filled disc of radius $r = 2$ pixels in a fixed RGB color. Integer dot positions are sampled uniformly in $[m, 64-1-m]^2$ with a margin of $m = r + \Delta_{\max} = 18$, and integer per-step displacements in $[-\Delta_{\max}, \Delta_{\max}]^2$ with $\Delta_{\max} = 16$, so every sampled transition stays inside the canvas. Rejection sampling enforces a minimum center-to-center separation of $2r + 1 = 5$ pixels at both endpoints. Each (o_t, a_t, o_{t+1}) is generated lazily from a seeded RNG, making the dataset deterministic and reproducible.

The four configurations of Fig. 4 combine three motion types: Independent dots have their own $(\Delta x, \Delta y)$ in the action vector, coupled dot pairs share a single $(\Delta x, \Delta y)$, and random dots move at every step without contributing to the action vector. Concretely, **Independent** = 2 independent dots ($|a| = 4$); **Coupled** = 1 coupled pair ($|a| = 2$); **Distractor** = 1 independent dot + 1 random dot ($|a| = 2$); **Combined** = 1 coupled pair + 2 independent dots + 1 random dot ($|a| = 6$).

E.2 Architecture and training

The encoder f_θ stacks three stride-2 convolutions ($3\to32\to64\to128$ channels, 3×3 kernels) interleaved with ReLU, followed by a 2×2 max-pool, a flatten, a linear projection to $\mathbb{R}^d$, and a final $\tanh$ that bounds embeddings in $[-1, 1]^d$ with $d = 64$. Both g_ϕ and h_ψ are 2-hidden-layer MLPs of width 256 with ReLU activations; g_ϕ ends in $\tanh$ to match the encoder range, while h_ψ has no output activation. Approximate parameter counts: $f_\theta \approx 220$k, $g_\phi \approx 100$k, $h_\psi \approx 100$k.

All three networks are trained jointly with Adam (lr $= 5\times10^{-4}$, default β, no weight decay, no LR schedule) for 100 epochs at batch size 256 on 250,000 transitions, with seed 42 and $\lambda = 10$. Actions are normalized by $\Delta_{\max}$ before being passed to g_ϕ and predicted by h_ψ, so all action coordinates lie in $[-1, 1]$; we report $\mathcal{L}_{\text{inv}}$ on these normalized actions. All metrics are computed on a held-out evaluation split of $1{,}000$ transitions sampled with a disjoint seed.

E.3 Collapse without the inverse loss

To verify that the inverse loss is the active anti-collapse mechanism on dot world, we rerun the single-dot configuration with $\lambda = 0$ and otherwise identical hyperparameters. Fig. 11 reports the resulting representation using the same probe grid as Fig. 2. Without the inverse loss, the encoder collapses all probe observations to essentially the same embedding. Consequently, there is no state-dependent variation for PCA to expose. The forward loss alone can therefore be minimized by a constant, trivially predictable representation.

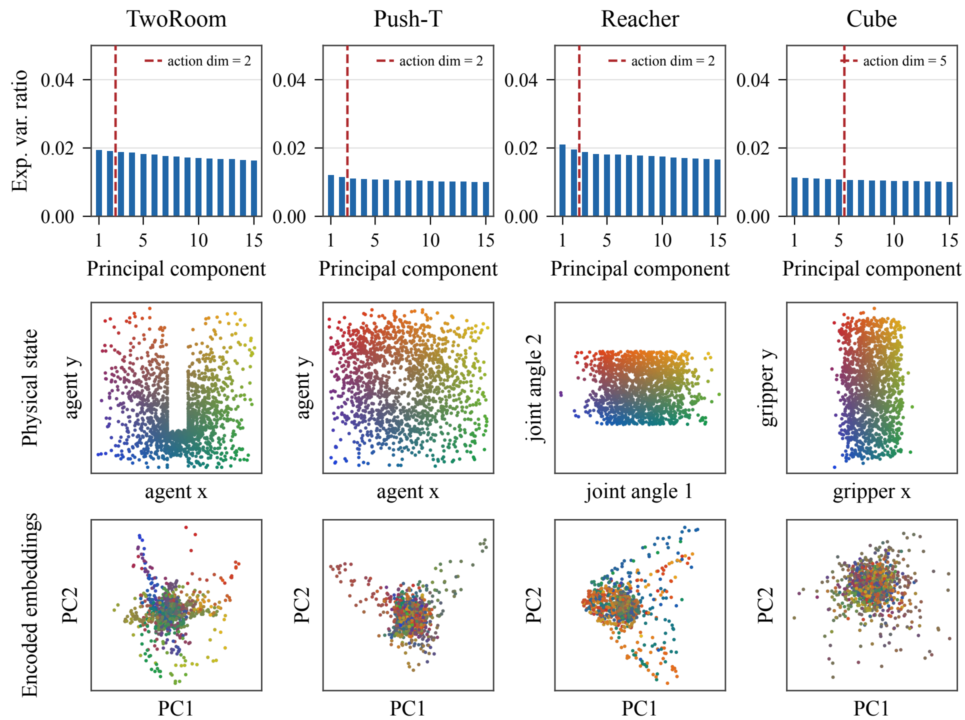

Figure 10: **Latent geometry of SIGReg embeddings.** For each environment we show the PCA spectrum of held-out embeddings (top), the distribution of a representative ground-truth quantity in physical state space (middle), and the first two principal components of the encoded embeddings, color-coded by the same physical quantity (bottom). The dashed red lines mark the action dimension. Compared with the inverse-dynamics-regularized embeddings in Fig. 6, SIGReg produces less compact and less structured latent geometry: the spectra lack a sharp elbow, and the state-dependent structure is spread across many components rather than appearing in a small interpretable PC subspace.

F Extended related work

This appendix expands the discussion of Sec. 2. Tab. 5 summarizes the discussed methods along the dimensions most relevant to our contribution.

F.1 Inverse dynamics for representation learning

F.1.1 Intrinsic Curiosity Module (ICM)

ICM [30] is the canonical inverse-dynamics-as-feature-learner construction we extend. Two architectural differences matter for our claim. First, in ICM the encoder receives gradients only from the inverse loss; the forward loss treats $\phi_\theta(o_{t+1})$ as a stop-gradient target and trains only g_ω. In our setup the encoder receives gradients from both losses, because the forward predictor is part of the world model we deliver. Second, ICM uses a separate A3C policy with its own encoder; the ICM encoder is used only to compute a scalar intrinsic reward. Our encoder *is* the world-model encoder used downstream. Conceptually, ICM motivates inverse dynamics as a way to ignore action-irrelevant variation, whereas we frame it as the anti-collapse mechanism that allows joint training of an end-to-end JEPA world model without reconstruction, EMA, or distributional regularizers.

F.1.2 Multi-step inverse identifiability theory

A line of theoretical work establishes asymptotic recovery of the control-endogenous latent state for multi-step inverse models in finite Exogenous Block MDPs [31–34]. We summarize their assumptions precisely here.

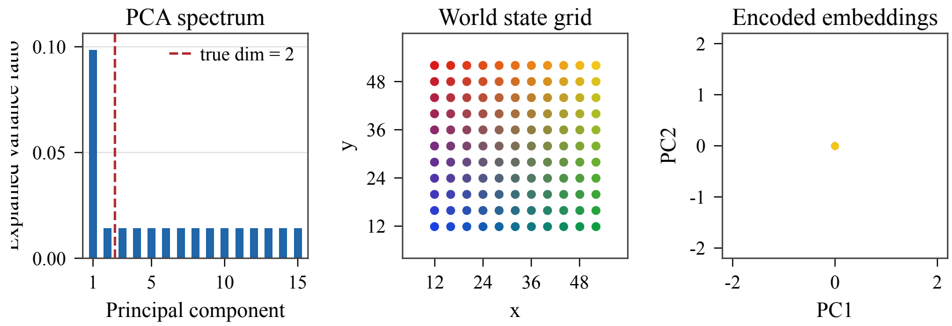

Figure 11: **Forward-only collapse on dot world.** The single-dot model is trained with $\lambda = 0$ and evaluated with the same probe grid and visualization protocol as Fig. 2. Removing the inverse loss maps the probe states to the same embedding, so the PC projection collapses to a single point at the origin.

Table 5: **Comparison of related methods.** “Inv.” and “Fwd.” denote inverse and forward dynamics components; “True action” indicates supervision with ground-truth actions; “WM” marks methods that deliver a world model usable for planning. The last column lists the primary anti-collapse mechanism. Our method is the only entry that combines offline reward-free world-model training with inverse dynamics as the sole anti-collapse mechanism.

Method	**Inv.**	**Fwd.**	**True action**	**Reward**	**Offline**	**WM**	**Anti-collapse**
ICM [30]	✓	✓	✓	×	×	×	Inv. dynamics on ϕ_θ
CURL [41]	×	×	✓	✓	×	×	Contrastive
Dreamer [4]	×	✓	✓	✓	×	✓	Recon. + KL
TD-MPC [5]	×	✓	✓	✓	×	✓	EMA latent target
DynaMo [39]	✓	✓	×	×	✓	✓	Joint inv. + fwd. dyn.
PLDM [13]	✓	✓	✓	×	✓	✓	VICReg + inv. dyn.
DINO-WM [12]	×	✓	✓	×	✓	✓	Frozen DINOv2
LeWorldModel [14]	×	✓	✓	×	✓	✓	SIGReg
WAM [35]	✓	✓	✓	✓	×	✓	Recon. + KL + inv. dyn.
EB-JEPA [36]	✓	✓	✓	×	✓	✓	VICReg + inv. dyn.
Ours	✓	✓	✓	×	✓	✓	Inv. dynamics alone

AC-State [32]. AC-State is proved correct under: (i) finite endogenous state space S; (ii) finite action space A; (iii) deterministic endogenous transition $T(s' \mid s, a)$; (iv) action-independent exogenous noise; (v) bounded endogenous diameter D and a behavior policy that mixes well; (vi) a multi-step inverse loss with horizon $K \geq D$, jointly with an information bottleneck that selects, among loss-minimizers, the encoder of minimum output cardinality. The conclusion is asymptotic identification of the partition induced by f^*. The authors explicitly note that the finite-state assumption “rules out continuous control problems”. Their Appendix B exhibits a discrete Ex-BMDP in which a *single-step* inverse model cannot distinguish two distinct endogenous states; Efroni et al. [31] and Islam et al. [33] provide further such counterexamples.

Refinements and known failure modes. Levine et al. [54] show that even AC-State can fail in deterministic Ex-BMDPs when the witness distance between two states exceeds D, and fails for any finite horizon K under periodic endogenous dynamics; they propose adding a latent forward-dynamics term to restore guarantees.

Implication for our work. Our setting differs from these results on every dimension that matters: continuous (rather than finite) state and action spaces, single-step (rather than multi-step) inverse, possibly stochastic endogenous dynamics, and no information bottleneck on the encoder’s output cardinality. We therefore do not claim that any of these guarantees transfer to our inverse-dynamics objective. We cite this body of work as conceptual motivation: a substantial theoretical literature confirms that inverse-dynamics objectives of various forms learn features tied to controllable structure under *some* conditions, and this informs our design choice. The single-step positive result of Brandfonbrener et al. [55] for a linear-Gaussian multitask imitation model is the closest theoretical endorsement of single-step inverse dynamics that we know of, but its scope (deconfounding latent task variables in imitation) does not match ours.

F.2 Inverse-dynamics-regularized world models

DynaMo [39]. DynaMo is architecturally the closest neighbor to ours: joint latent inverse and forward dynamics without reconstruction. The decisive difference is action supervision: DynaMo treats actions as unobserved latents inferred jointly with the encoder, while we supervise the inverse head with the executed action a_t. Ground-truth labels turn the inverse term into a hard anti-collapse signal that a co-learned action representation cannot provide.

LAPO and APV. Inverse-dynamics ideas also underlie work that *infers* pseudo-actions from action-free video. LAPO [38] uses inverse dynamics model to discover latent actions and trains a forward model conditioned on these. APV [40] pretrains an action-free latent video predictor and fine-tunes for control. These methods address the complementary regime where ground-truth actions are unavailable; with actions in hand, we use them directly to supervise inverse-dynamics prediction.

World-Action Model [35]. WAM augments DreamerV2 with an inverse dynamics head and reports that this regularizes the latent state toward action-relevant structure. The construction is conceptually closest to ours among reconstruction-based methods. Two differences are decisive: WAM operates within Dreamer's pixel-reconstruction ELBO and requires reward supervision; we are reconstruction-free, JEPA-style, and reward-free. WAM treats the inverse dynamics head as one auxiliary among many; we identify it as the sufficient anti-collapse mechanism for end-to-end JEPA training.

EB-JEPA [36]. EB-JEPA is an action-conditioned JEPA with a multi-term loss combining variance, covariance, temporal-similarity, and inverse-dynamics regularizers; their ablations report that all four terms contribute to anti-collapse. Our claim is more specific: the inverse-dynamics term *alone* suffices, removing the need for variance and covariance terms and yielding a substantially simpler training objective.

Auxiliary-task JEPA theory [37]. Concurrent theoretical work proves a no-unhealthy-collapse theorem for JEPAs with an auxiliary regression head: in deterministic MDPs, if both the latent prediction loss and the auxiliary regression loss are zero, then observations with different transition dynamics or different auxiliary values must map to distinct embeddings. Our IDR objective is the action-prediction instantiation of this template. Our contribution complementary to theirs is empirical: we instantiate the template on continuous pixel-action data and show that single-step IDR is sufficient in practice.

F.3 Alternative anti-collapse mechanisms in latent world models

F.3.1 Reconstruction-based

The Dreamer family [2, 4, 24] and PlaNet [25] train recurrent state-space models with pixel-reconstruction and reward-prediction heads under a variational objective:

$$\mathcal{L}_{\text{Dreamer}} = \mathbb{E}\left[\sum_t -\log p_\theta(o_t|h_t, z_t) - \log p_\theta(r_t|h_t, z_t) + \beta\,\mathrm{KL}\big(q_\theta(z_t|\cdot)\,\|\,p_\theta(z_t|h_t)\big)\right]. \quad (7)$$

DreamerV3 retains this objective family but adds engineering refinements (symlog targets, KL balancing, two-hot value targets) for stability across domains. Reconstruction provides an implicit anti-collapse signal but commits modelling capacity to pixel-level details that are often irrelevant for control. Our setting differs from Dreamer's in two ways: we train without reward labels, and we predict in embedding space rather than reconstructing observations.

F.3.2 EMA latent targets

TD-MPC [5, 6] learns a latent encoder $z_t = \phi_\theta(o_t)$ and a latent transition g_ω together with reward and value heads, all trained end-to-end via TD learning. The objective combines reward, value, and a latent-consistency term that regresses $g_\omega(z_t, a_t)$ onto $\texttt{sg}(\phi_{\theta^-}(o_{t+1}))$, where θ^- is an EMA copy of θ. SPR [56] applies the same idea to data-efficient Atari, predicting k-step latent rollouts against a BYOL-style [10] momentum target encoder. Both methods rely on the EMA target as the primary defense against collapse: a slowly-moving target makes constant solutions a moving target. TD-MPC additionally requires reward supervision. Our work avoids EMA targets and reward labels, replacing both with the inverse-dynamics signal.

F.3.3 Augmentation and contrastive methods

CURL [41] adds an InfoNCE [26] contrastive auxiliary loss between augmented views of the same observation, sharing the encoder with the policy and Q-function. DrQ [42] shows that random image-shift augmentation alone, without an explicit auxiliary loss, suffices for state-of-the-art visual RL. Proto-RL [43] uses prototypical clustering of embeddings as both an exploration bonus and a representation learner. None of these methods learns a world model in our sense: they learn encoders shared with model-free policies. Their anti-collapse mechanisms (negatives, augmentation, prototype balance) are orthogonal to inverse-dynamics regularization and could in principle be combined with it.

F.3.4 JEPA-style regularizers

JEPAs predict in embedding space, with the target embedding produced by an encoder applied to the future or masked observation, typically tied to the online encoder via stop-gradient and EMA [7–9]. In the action-conditioned regime relevant to control, several recent methods stand out. Zhou et al. [12] (DINO-WM) freeze a pretrained DINOv2 encoder and learn only a latent dynamics model on top. Sobal et al. [13] (PLDM) train end-to-end with a multi-term objective combining VICReg-style regularizers [11] with an inverse-dynamics term. Maes et al. [14] (LeWorldModel) build on LeJEPA [15] and regularize embeddings toward isotropic Gaussianity using SIGReg. Assran et al. [16] (V-JEPA 2) scale to large internet-video corpora and rely on EMA targets and a separate action-conditioning post-training stage. Our inverse-dynamics objective adds a distinct point on this design axis: a task-grounded action-prediction signal anchoring the embedding to controllable structure.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction claim that single-step inverse dynamics regularization (i) prevents collapse in end-to-end JEPA-style latent world models and (ii) yields compact, action-aligned representations supporting latent planning. Both claims are supported in Secs. 4 and 5. The empirical scope is restricted to moderate-scale simulated control tasks, as stated in Sec. 6.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations are discussed in Sec. 6, including action recoverability, single-frame encoders, biased behavior policies, offline coverage, compounding model error, and the moderate scale of the simulated tasks.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no formal theoretical results. Prior identifiability theorems for multi-step inverse models are cited only as conceptual motivation (Sec. 2), and we explicitly state that they do not transfer to our setting.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The method, loss, pseudocode, architecture, optimization settings, planning protocol, datasets, and train/validation split are specified in Secs. A to 3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
    (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
    (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
    (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Anonymized code supplements are provided for the method and evaluations. The datasets and environment code are public assets cited in Secs. A and B.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: All the training and test details are given in Secs. A, A.4 and B.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Planning success in Figs. 5 and 9 is reported as mean $\pm$ standard error of the mean over five training seeds, on a fixed set of 100 tasks per environment shared across methods and seeds. The error bars therefore capture variability due to random initialization and training stochasticity.

   Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Sec. A reports the hardware used and an upper-bound on the per-model training time.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses simulated control environments and public research code/datasets, does not involve human subjects, and does not release high-risk models.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The work is foundational research on latent world models and is not tied to a deployed application. We do not identify a direct negative societal impact beyond general dual-use concerns for improved control methods.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released assets are small research models for simulated control environments.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing environments, datasets, libraries, and baselines are cited in Secs. A, B and F. URLs and licenses are provided in the supplementary material.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The released asset is an anonymized code supplement documenting training and evaluation.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

Justification: The work does not involve human subjects and therefore does not require IRB approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not part of the method, experiments, or scientific contribution. Any use was limited to writing, editing, formatting, or code-assistance support.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.